PROJECT REPORT

Metro Ticket Generating System

Digital metro ticket booking with automatic fare calculation and QR code tickets, built on ServiceNow

Project date: 15 October 2026

Contents

1. Introduction

2. Ideation Phase

3. Requirement Analysis

4. Project Design

5. Project Planning & Scheduling

6. Functional and Performance Testing

7. Results

8. Advantages & Disadvantages

9. Conclusion

10. Future Scope

11. Appendix

1. INTRODUCTION

1.1 Project Overview

Metro Ticket Generating System is a ServiceNow application that digitises metro ticket booking. A passenger opens the ServiceNow Service Portal, fills a short form called 'Book A Metro Ticket', and the system calculates the fare automatically, shows a QR code ticket and creates a request number. Every booking is then saved in a custom table and a verification task is sent to the Station Managers group.

The solution was built on a ServiceNow Personal Developer Instance (PDI) in the Global scope using low-code platform features: custom tables, a Service Catalog item with 8 variables, catalog client scripts, a catalog UI policy, a Service Portal widget, Flow Designer and the platform's default access controls (ACLs). Stations from the Hyderabad metro (Ameerpet, Madhapur, LB Nagar, Kukatpally and Jubilee Hills) were used as sample data.

1.2 Purpose

The purpose of the project is to replace slow, paper-based ticketing with a fast, accessible and automated digital process. It supports five business objectives:

- **Enhance commuter convenience** – book online from a phone or computer and get a QR ticket without standing in a queue.
- **Increase operational efficiency** – the fare is calculated automatically, removing manual work and errors at the counter.
- **Promote digital payments** – passengers choose UPI, card or another cashless mode.
- **Facilitate data-driven decisions** – every booking is stored in one table, ready for analysis of routes and busy times.
- **Support environmental sustainability** – digital QR tickets replace paper tokens.

2. IDEATION PHASE

2.1 Problem Statement

Metro passengers wait in long queues to buy paper tickets, must carry exact change, and often do not know the fare in advance. Station staff have no single digital record of bookings, so verifying tickets and studying travel patterns is difficult.

PS	I am	I'm trying to	But	Because	Which makes me feel
PS-1	A daily metro commuter	Buy a ticket quickly before my train	I stand in long queues and need exact change	Tickets are manual / paper and fares are on a chart	Frustrated, stressed and late
PS-2	A metro station manager	Verify tickets and know sales per route	Paper and cash records are scattered	No single digital system records bookings	Overloaded, unable to decide with data

2.2 Empathy Map Canvas

The empathy map below describes a daily metro commuter: what they think and feel, hear, see, say and do, along with their pains and gains.

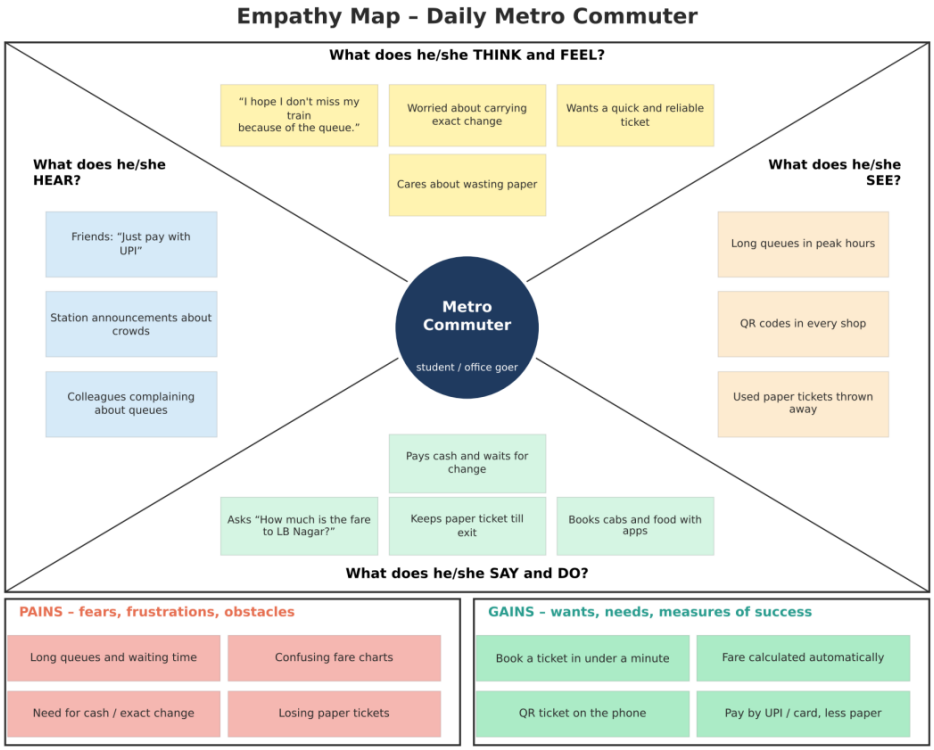


Figure 1: Empathy map – daily metro commuter

- Commuters lose time in queues and worry about carrying change.
- They already trust UPI and QR codes in daily life.
- They want to know the fare before travelling.
- Station managers need one place to see and verify every booking.

2.3 Brainstorming

The team (Team Members 1–4) held an online whiteboard session. Ideas were listed and grouped into six themes, then prioritised by importance and feasibility. The 'Do now' ideas became the Phase 1 scope.

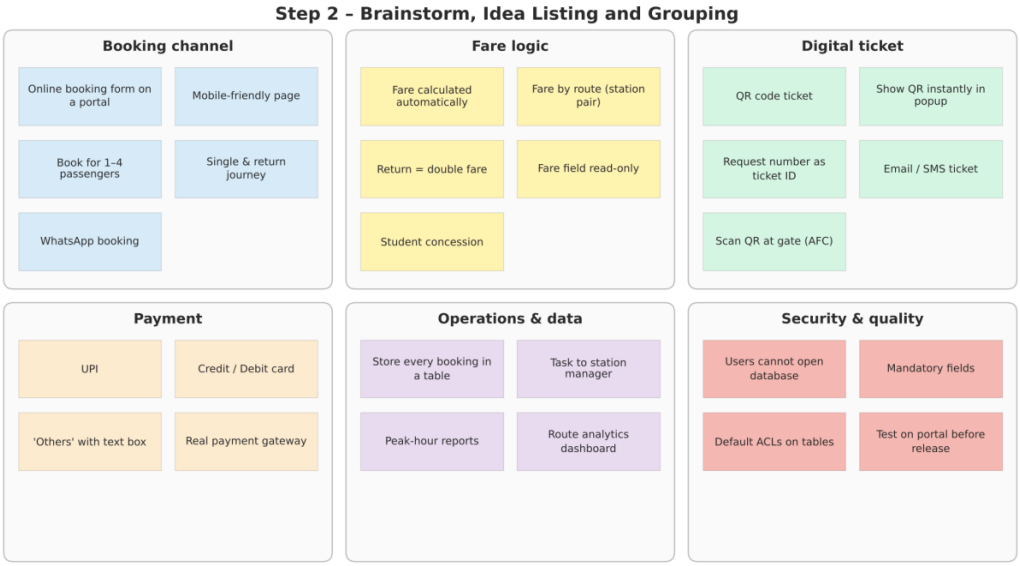


Figure 2: Brainstorm – idea listing and grouping

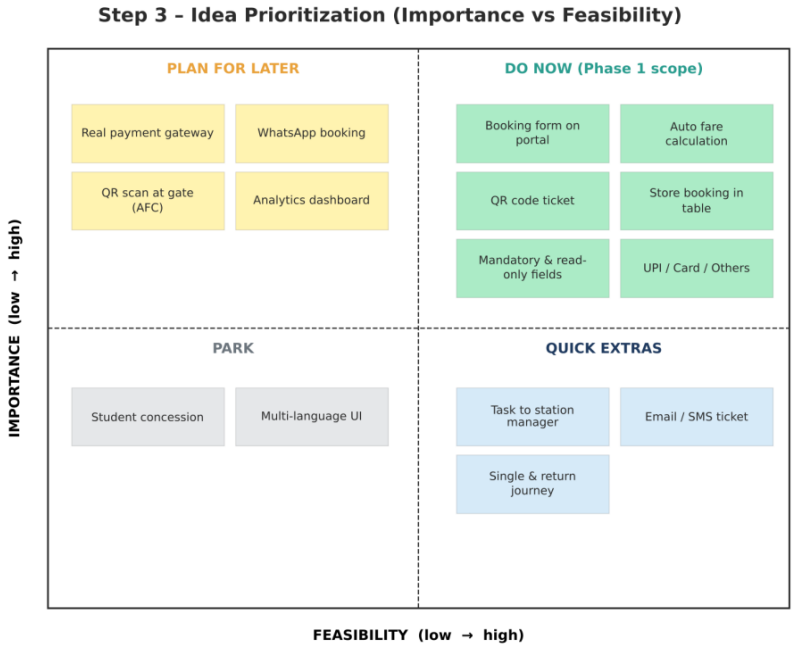


Figure 3: Idea prioritisation – importance vs feasibility

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The customer journey map follows a passenger through five stages – Entice, Enter, Engage, Exit and Extend – showing the steps, interactions, goals, positive and negative moments, and opportunities at each stage.

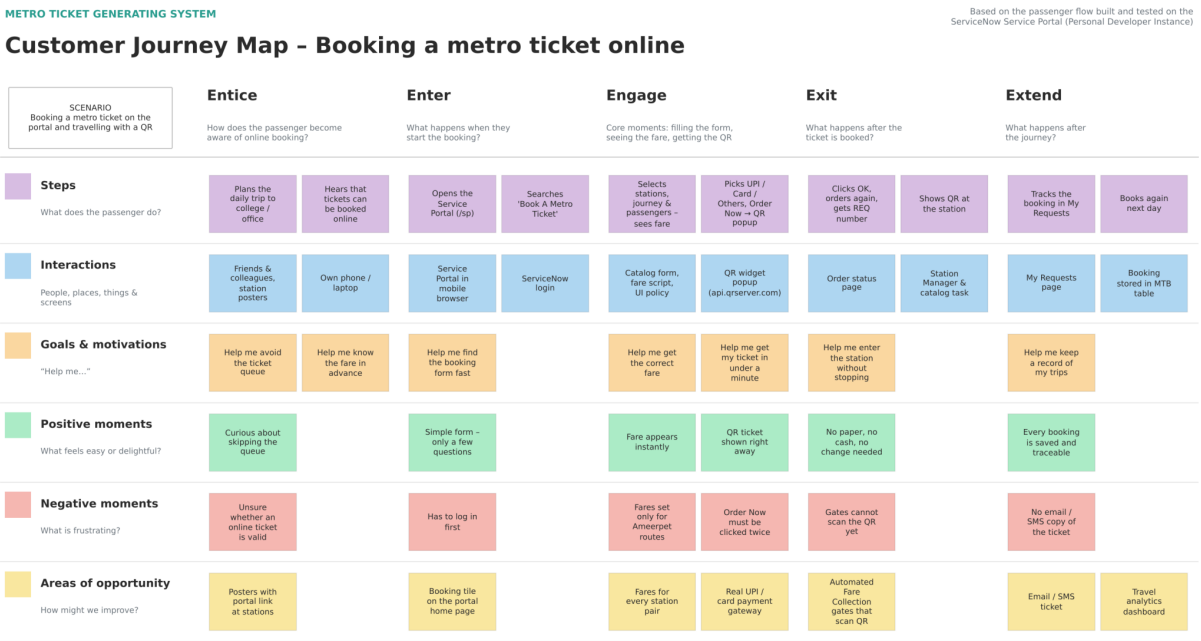


Figure 4: Customer journey map – booking a metro ticket online

3.2 Solution Requirements

Functional requirements

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Ticket Booking	'Book A Metro Ticket' catalog item on the Service Portal; select stations, journey type (Single/Return) and 1–4 passengers
FR-2	Station Master Data	Stations stored in Metro Station's Details (u_metro_station_details); Station Name is the display value
FR-3	Automatic Fare Calculation	Client script: fare = route fare × passengers (× 2 for return); amount fields read-only
FR-4	Payment Mode Selection	UPI / Card / Others; UI policy shows mandatory 'Enter payment mode' for Others
FR-5	QR Code Ticket	onSubmit script builds the QR via QR Code API and shows it in metro_qr_widget
FR-6	Booking Storage & Verification	REQ/RITM created; flow stores the booking (MTB) and creates a task for Station Managers

Non-functional requirements

NFR No.	Requirement	Description
NFR-1	Usability	Simple portal form with 8 fields; fare fills automatically; works on desktop and mobile browsers
NFR-2	Security	Default ACLs on custom tables; users book only through the catalog; flow runs as System
NFR-3	Reliability	Mandatory and read-only fields prevent wrong bookings; each request has a unique REQ/RITM
NFR-4	Performance	Fare appears instantly in the browser; QR appears within seconds; flow runs right after submission
NFR-5	Availability	Hosted on the ServiceNow cloud, available 24×7
NFR-6	Scalability	New stations and fares added as data; more features can be added on the same platform

3.3 Data Flow Diagram

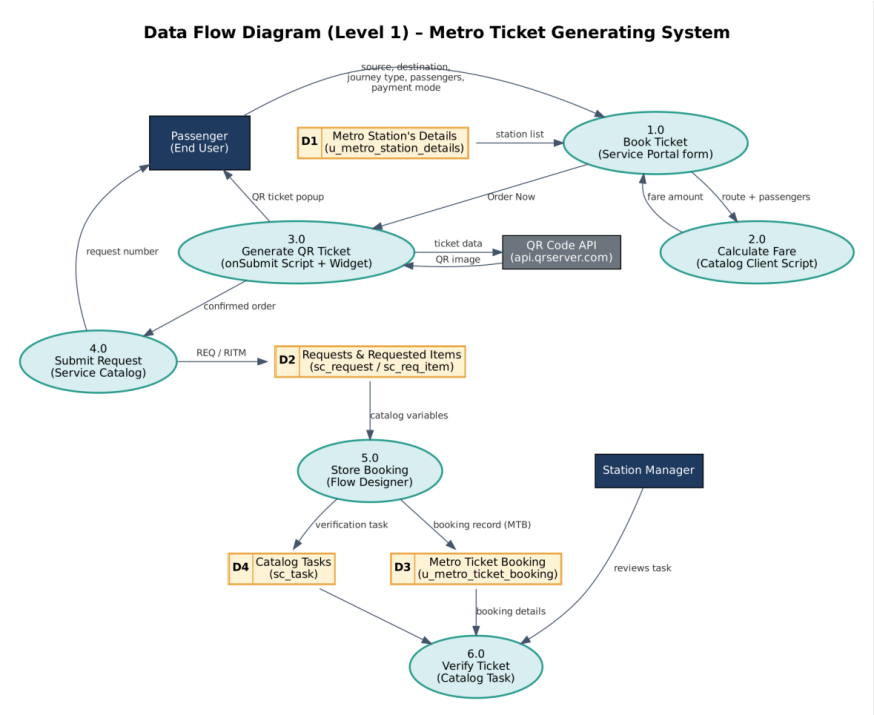


Figure 5: Data flow diagram (Level 1)

The passenger's journey details flow into the booking form (1.0). The fare script (2.0) returns the amount. The QR script (3.0) exchanges ticket data with the QR Code API. The submitted request (4.0) is stored in the request tables, and the flow (5.0) saves the booking (D3) and creates a verification task (D4) that the Station Manager completes (6.0).

User stories

User Type	Epic	USN	User Story	Priority	Release
Passenger	Ticket Booking	USN-1	Find 'Book A Metro Ticket' on the Service Portal	High	Sprint-1
Passenger	Ticket Booking	USN-2	Select Starting From and Going To stations	High	Sprint-1
Passenger	Ticket Booking	USN-3	Choose Single/Return journey and 1–4 passengers	High	Sprint-1
Passenger	Fare Calculation	USN-4	See the fare calculated automatically	High	Sprint-2
Passenger	Payment	USN-5	Select UPI / Card / Others and enter the mode for Others	Medium	Sprint-2
Passenger	QR Ticket	USN-6	See a QR ticket before confirming the order	High	Sprint-2
Passenger	Request Tracking	USN-7	Get a REQ number and track it in My Requests	High	Sprint-2
Station Manager	Verification	USN-8	Receive a task to verify each new ticket	Medium	Sprint-3
Operations Team	Booking Data	USN-9	See every booking stored in one table	High	Sprint-3
IT Admin	Security	USN-10	Ensure normal users cannot open the tables	High	Sprint-4
IT Admin	Testing	USN-11	Test the full booking on the portal	Medium	Sprint-4
IT Admin	Station Data	USN-12	Add or update stations in the station table	Low	Sprint-1

3.4 Technology Stack

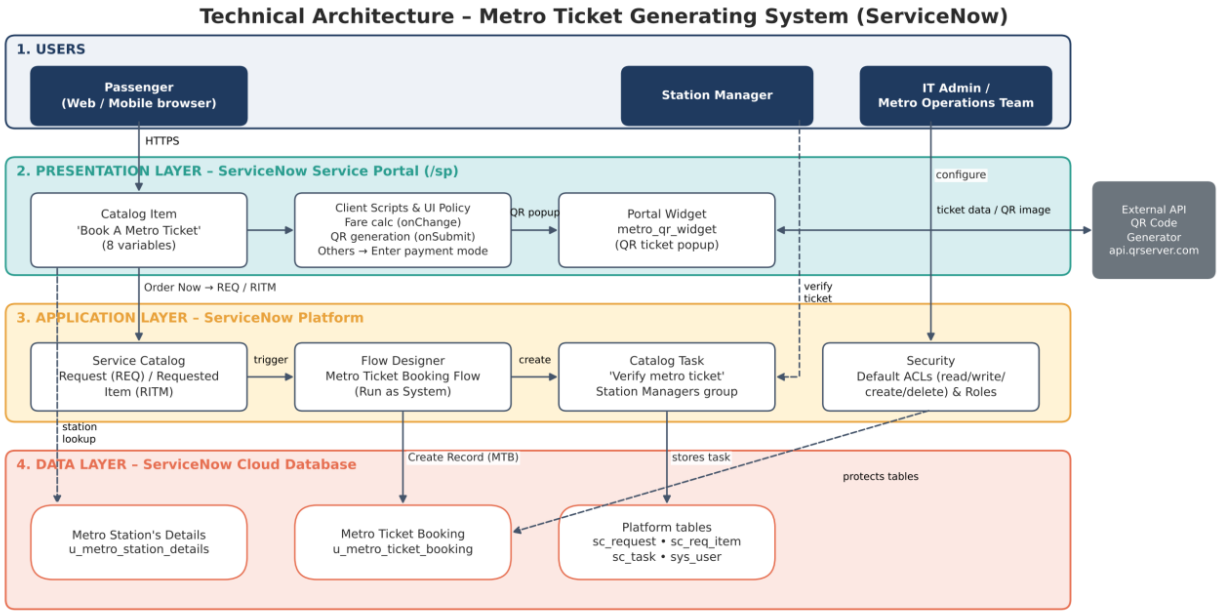


Figure 6: Technical architecture

Component	Description	Technology
User Interface	Booking form and QR popup in the browser	ServiceNow Service Portal, Service Catalog, AngularJS widget
Application Logic-1	Fare calculation	Catalog Client Script (onChange, JavaScript)
Application Logic-2	QR generation; payment-mode visibility	Catalog Client Script (onSubmit), spModal, Catalog UI Policy

Component	Description	Technology
Application Logic-3	Store booking, create verification task	Flow Designer
Database	Stations and bookings	ServiceNow tables u_metro_station_details, u_metro_ticket_booking
Cloud Database	Requests, tasks, users	sc_request, sc_req_item, sc_task, sys_user
External API	QR image generation	QR Code Generator API (api.qrserver.com)
Infrastructure	Cloud hosting	ServiceNow Personal Developer Instance
Security	Access control	Default ACLs, roles, HTTPS

4. PROJECT DESIGN

4.1 Problem Solution Fit

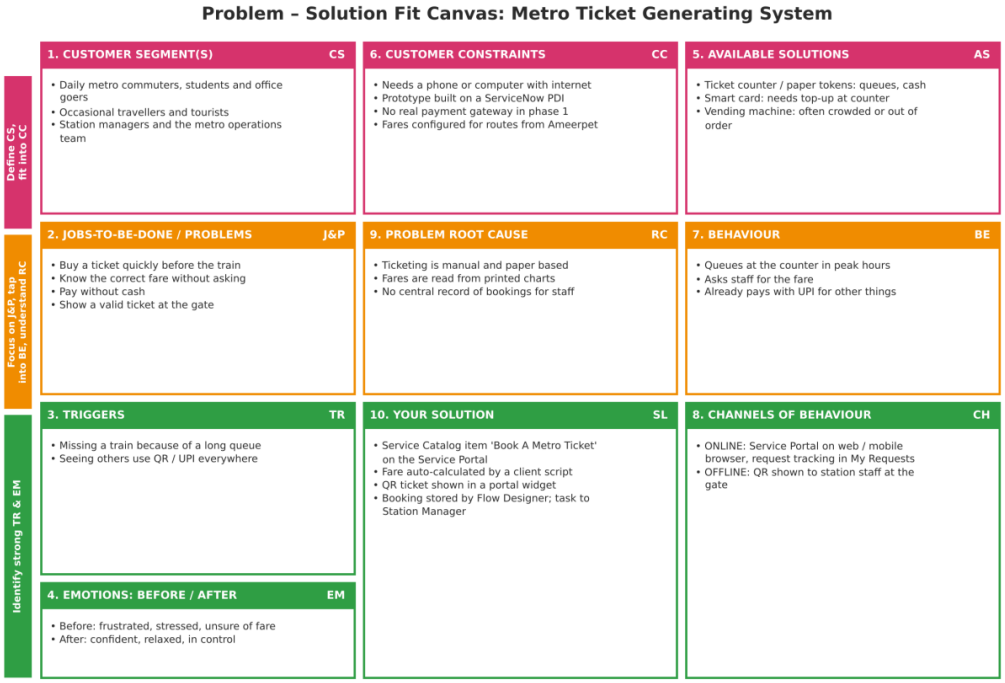


Figure 7: Problem – solution fit canvas

Commuters lose time in queues and do not know the fare in advance because ticketing is manual. They already use UPI and QR codes every day, so a self-service portal that calculates the fare and shows a QR ticket instantly fits their habits and removes the main frustrations.

4.2 Proposed Solution

Parameter	Description
Problem Statement	Long ticket queues, cash handling, unknown fares and no digital booking record for staff.
Idea / Solution	ServiceNow catalog item for booking with automatic fare, QR ticket in a portal widget, request tracking, booking storage by Flow Designer and a verification task for Station Managers.
Novelty / Uniqueness	Built fully on a low-code enterprise platform – no separate servers; booking, logic, data, tasks and security in one place; new routes added as data.
Social Impact	Shorter queues, cashless choice, paperless tickets and a traceable record of every booking.
Business Model	Saves printing and counter costs; future convenience fee, payment partners, advertising and monthly passes.
Scalability	More stations, lines and fares as records; payment gateway, SMS/email tickets, AFC gates and dashboards on the same platform.

4.3 Solution Architecture

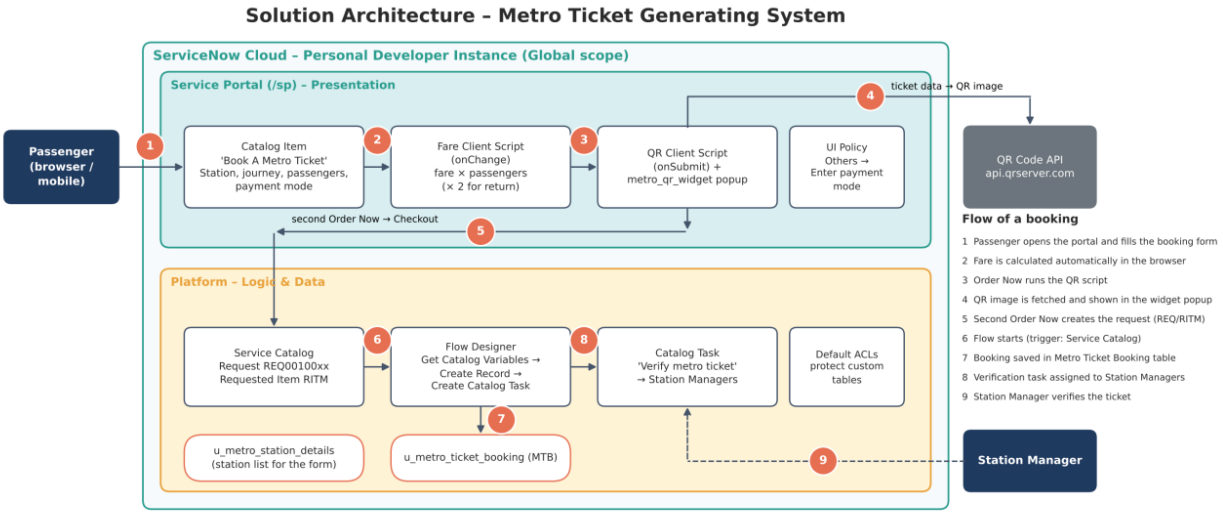


Figure 8: Solution architecture and data flow

- **Presentation:** catalog item 'Book A Metro Ticket' with 8 variables on the Service Portal; stations come from Metro Station's Details.
- **Fare logic:** onChange client script on No of Passengers fills the read-only amount field.
- **Payment rule:** catalog UI policy shows 'Enter payment mode' only for Others.
- **QR ticket:** onSubmit client script calls the QR Code API and opens metro_qr_widget; the flag g_form.metroQrDone lets the second Order Now place the order.
- **Automation:** 'Metro Ticket Booking Flow' (run as System) – Get Catalog Variables → Create Record (MTB) → Create Catalog Task.
- **Security:** default ACLs stop normal users from opening the tables.

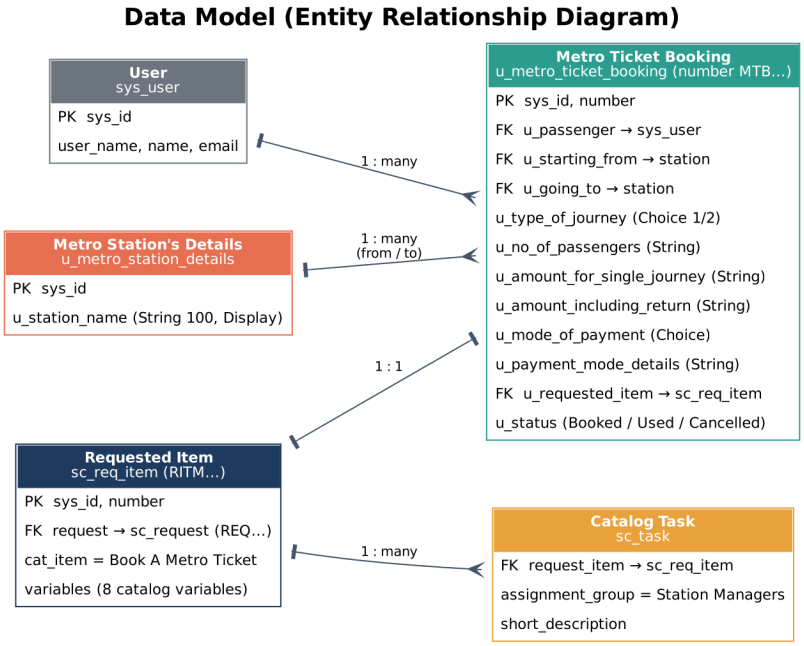


Figure 9: Data model (entity relationship diagram)

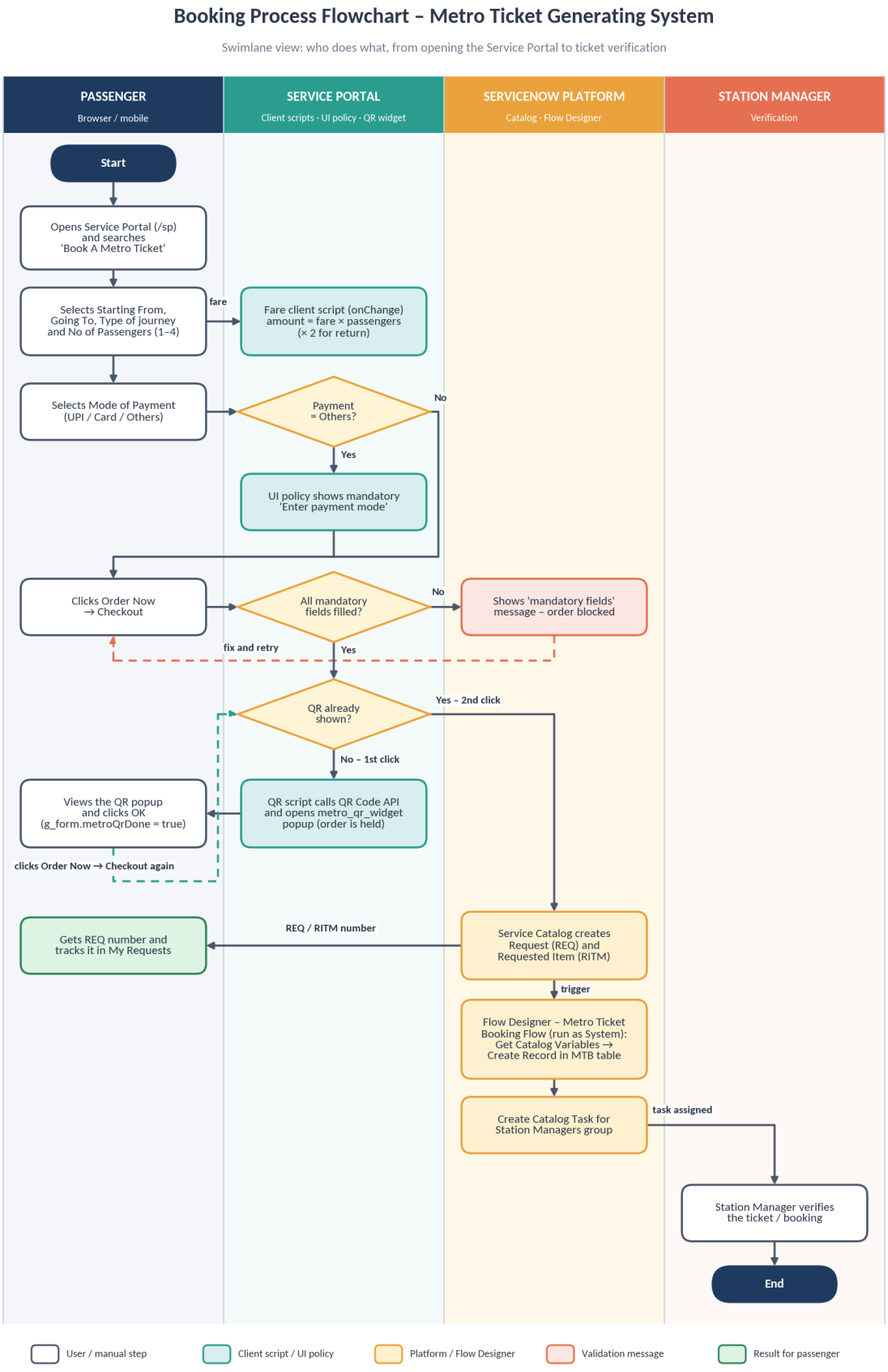


Figure 10: Booking process flowchart (swimlane view)

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project followed an agile approach with four sprints and these milestones: Catalog Form Setup → Fare Logic Script → QR Code Integration → Station Details Mapping → UI Pages → UAT & Deployment. 16 user stories (43 story points) were planned and completed between 08 and 14 October 2026, and the project documentation was finalised on 15 October 2026.

Sprint	Epics	Story Points	Duration	Start	End	Completed
Sprint-1	Project Setup & Data Architecture; Catalog Form Setup	9	2 Days	08 Oct 2026	09 Oct 2026	9
Sprint-2	Fare Logic; QR Code Integration	15	2 Days	10 Oct 2026	11 Oct 2026	15
Sprint-3	Booking Storage & Automation	9	2 Days	12 Oct 2026	13 Oct 2026	9
Sprint-4	Security & UAT; Documentation & Demo	10	1 Day	14 Oct 2026	14 Oct 2026	10
Total	7 epics, 16 user stories	43	7 Days	08 Oct 2026	14 Oct 2026	43

Velocity: 43 story points ÷ 4 sprints = 10.75 story points per sprint. Average velocity per day = 43 ÷ 7 ≈ 6.1 story points per day.

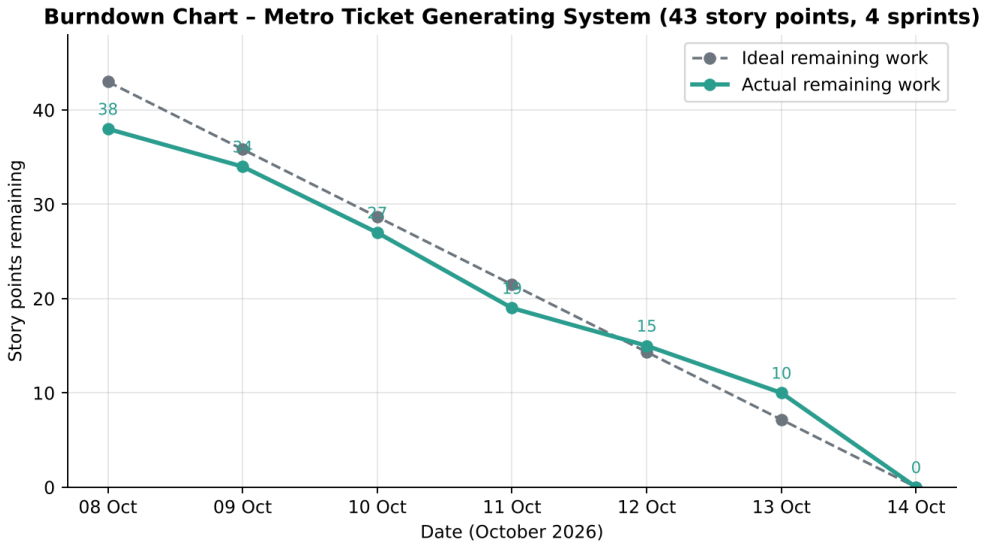


Figure 11: Burndown chart

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Functional and performance tests were run on the Service Portal with test bookings.

ID	Scenario	Expected Result	Result
FT-01	Text input – Enter payment mode	Mandatory for Others; valid text accepted	Pass
FT-02	Number input – No of Passengers	Only 1–4; empty value blocked	Pass
FT-03	Fare and QR generation	Amount 100 for Ameerpet → LB Nagar (2); QR shown	Pass
FT-04	QR Code API connection	QR image loads and can be scanned	Pass
PT-01	Response time	Fare and QR appear in under 3 seconds	Pass
PT-02	Repeated bookings	Each booking gets QR, REQ and a completed flow	Pass

ID	Scenario	Expected Result	Result
PT-03	File upload load	Not applicable – no uploads in the form	N/A

User Acceptance Testing

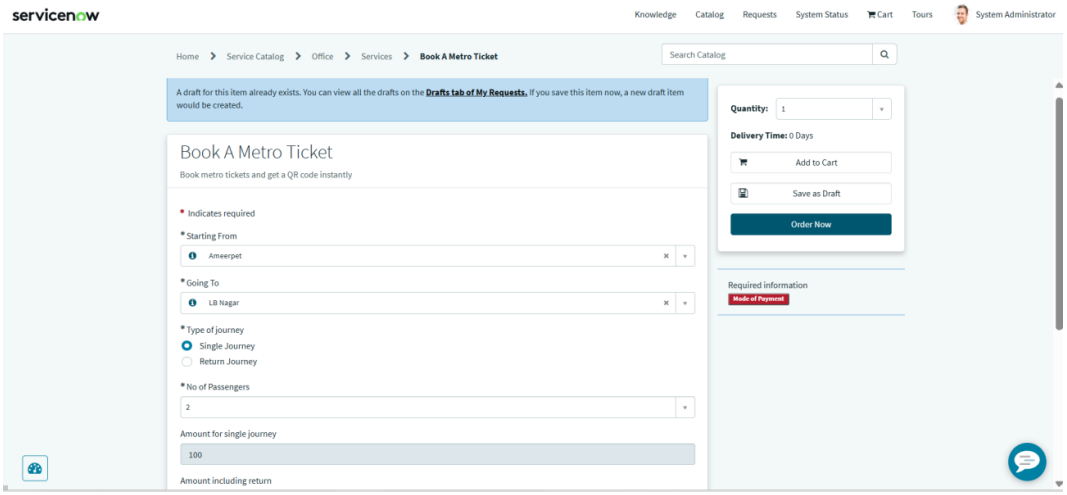
15 UAT test cases covering station data, form behaviour, fare calculation, UI policy, QR generation, request submission, flow and security were executed – all 15 passed. 8 defects were logged during development: 6 were closed and 2 remain as known limitations.

Bug ID	Issue found	Resolution
BG-001	Fare script gave a syntax error (extra closing bracket '}' in the guide's script)	Removed the extra bracket
BG-002	Fare not calculated – script used station sys_ids from another instance	Replaced with own station sys_ids
BG-003	Flow could not map No of Passengers (Select Box value vs Integer column)	Changed column type to String
BG-004	Order never submitted – QR script always returned false, so no REQ number	Added a check for the 'QR shown' flag
BG-005	JavaScript error: Cannot read properties of null (reading 'metroQrDone') on the portal	Stored the flag on g_form instead of window
BG-006	Both amount fields are visible before a journey type is selected	Kept as in the project guide
BG-007	Amount does not update when stations or journey are changed after choosing passengers	Workaround: choose No of Passengers last
BG-008	Routes without a configured fare leave the amount empty	Add fares for all station pairs in future

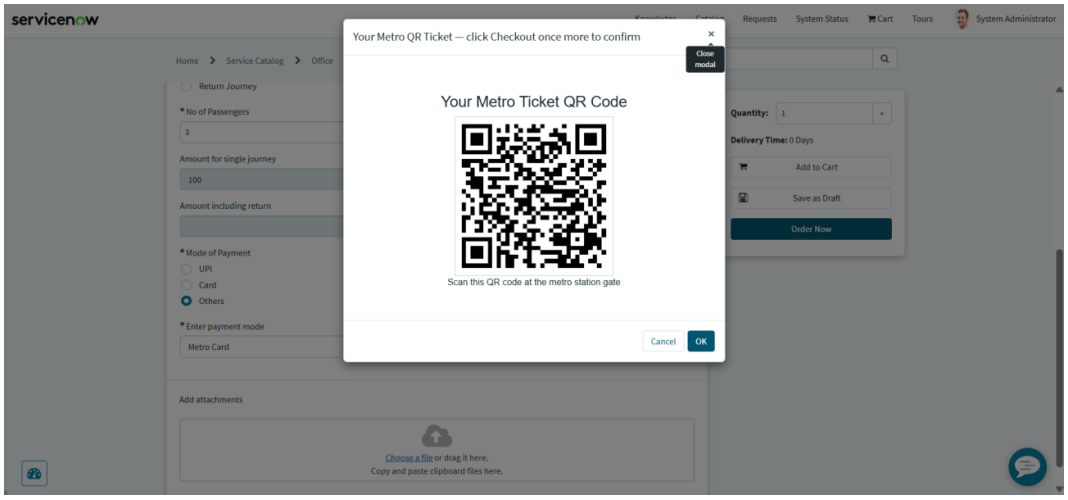
7. RESULTS

7.1 Output Screenshots

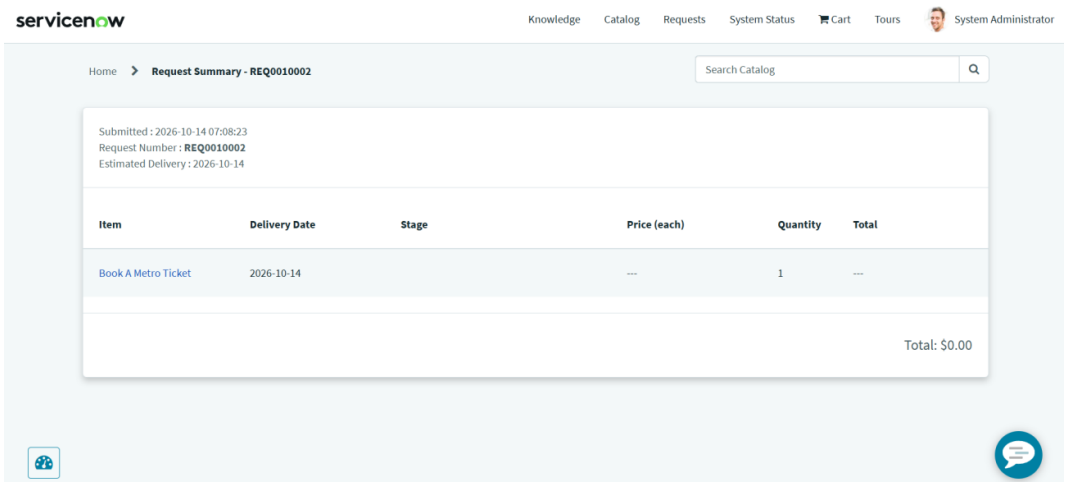
The system works end to end: the passenger books a ticket on the Service Portal, the fare is calculated automatically, a QR ticket is displayed, a request number is generated, the booking is stored in the Metro Ticket Booking table and a verification task is created for Station Managers.



Output 1: Booking form with fare calculated automatically



Output 2: QR code ticket shown in the popup



Output 3: Request submitted with request number

8. ADVANTAGES & DISADVANTAGES

Advantages

- Tickets are booked in under a minute without standing in a queue.
- Fare is calculated automatically, so there are no manual errors.
- QR ticket is paperless and eco-friendly.
- Cashless payment modes (UPI, card, others) are supported.
- Every booking is stored in one table and can be tracked with a request number.
- Station Managers automatically receive a verification task.
- Built on low-code ServiceNow features – easy to maintain and extend; default ACLs keep data secure.

Disadvantages

- Fares are configured only for routes from Ameerpet in this version.
- The fare updates only when No of Passengers is changed, so passengers must select it last.
- Order Now has to be clicked twice (first click shows the QR ticket).
- No real payment is processed – the payment mode is only recorded.
- The QR code holds a ticket link, and station gates cannot scan it automatically yet.
- Requires internet access and a ServiceNow login.

9. CONCLUSION

The Metro Ticket Generating System shows that metro ticket booking can be digitised quickly on the ServiceNow platform. Using a catalog item, client scripts, a UI policy, a portal widget, Flow Designer and default ACLs, the team built a working solution where passengers book tickets online, see the correct fare instantly, receive a QR code ticket and get a request number, while every booking is stored and passed to station staff for verification. All 15 UAT test cases passed. The project meets its goals of commuter convenience, operational efficiency, digital payment adoption, data collection for decision making and paper reduction, and provides a strong base for future features.

10. FUTURE SCOPE

- Payment gateway integration for real UPI, card and wallet payments.
- SMS, email and WhatsApp delivery of the QR ticket.
- Automated Fare Collection (AFC) gates that scan and validate the QR code.
- Fares for every station pair and multiple metro lines; concessions for students and senior citizens.
- Travel analytics dashboard showing peak hours, busiest routes and payment-mode share.
- A mobile app and multi-language support.

11. APPENDIX

Source Code

A. Fare calculation – Catalog Client Script (onChange on no_of_passengers)

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') { return; }
    var source = g_form.getValue('starting_from');
    var destination = g_form.getValue('going_to');
    var numPassengers = Number(newValue);
    var journeyType = g_form.getValue('type_of_journey');
    var AMEERPET = 'a7cc5f4683b60b143f98f065eeaad356';
    // destination sys_id -> [single fare, return fare] per passenger
    var fares = {
        'f4ec9f4683b60b143f98f065eeaad37a': [30, 60],   // Madhapur
        'f556f41e83724b143f98f065eeaad3e3': [50, 100],  // LB Nagar
        '0246305683b24b143f98f065eeaad34e': [30, 60],   // Kukatpally
        '1136fc1683b24b143f98f065eeaad33f': [20, 40]    // Jubilee Hills
    };
    g_form.clearValue('amount_for_single_journey');
    g_form.clearValue('amount_including_return');
    if (source != AMEERPET || !fares[destination]) { return; }  // no route matched
    if (journeyType == '1') {
        g_form.setValue('amount_for_single_journey', numPassengers * fares[destination][0]);
    } else if (journeyType == '2') {
        g_form.setValue('amount_including_return', numPassengers * fares[destination][1]);
    }
}
```

B. QR code generation – Catalog Client Script (onSubmit, Mobile / Service Portal)

```
function onSubmit() {
    // second click: QR already shown, so allow the order
    if (g_form.metroQrDone) { g_form.metroQrDone = false; return true; }
    var sysId = g_form.getUniqueValue();
    var dummyURL = 'https://example.com/ticket?id=' + sysId;
    var qrImageUrl = 'https://api.qrserver.com/v1/create-qr-code/?size=250x250&data='
                     + encodeURIComponent(dummyURL);
    spModal.open({
        title: "Your Metro QR Ticket - click Order Now once more to confirm",
        widget: "metro_qr_widget",
        widgetInput: { qr_url: qrImageUrl }
    }).then(function() { g_form.metroQrDone = true; });   // remember that the QR was shown
    return false;                                         // block only the first Order Now click
}
```

C. Metro QR Widget – HTML Template and Server Script

```
<div style="text-align:center; padding:25px; font-family:Arial, sans-serif;">
  <h3>Your Metro Ticket QR Code</h3>
  <img ng-src="{{data.qr_url}}" style="width:250px; height:250px; border:1px solid #ccc; padding:10px;" />
  <p>Scan this QR code at the metro station gate</p>
</div>
```

```
(function() {
    data.qr_url = input ? input.qr_url : '';
})();
```

D. Catalog UI Policy

Condition: Mode of Payment is Others (On load, Reverse if false). Action: enter_payment_mode – Visible = True, Mandatory = True.

E. Flow Designer – Metro Ticket Booking Flow (Run as System)

Trigger: Service Catalog → 1. Get Catalog Variables (Book A Metro Ticket) → 2. Create Record in Metro Ticket Booking (variables mapped to fields) → 3. Create Catalog Task (assignment group Station Managers).

Dataset Link

No external dataset is used. Station data is stored in the ServiceNow table Metro Station's Details (u_metro_station_details). Fares used in this version:

Route (either direction)	Single journey per passenger (₹)	Return per passenger (₹)
Ameerpet – LB Nagar	50	100
Ameerpet – Madhapur	30	60
Ameerpet – Kukatpally	30	60
Ameerpet – Jubilee Hills	20	40

GitHub & Project Demo Link

GitHub Link: https://github.com/NCharanTeja/Metro-Ticket-Generating-System

Project demo video:
https://drive.google.com/file/d/1AtEF1eX5mBMuy5Oe_T1a0HMlIhbBOS1s/view?usp=sharing